ENTERPRISE DOCUMENTATION

CI/CD Pipeline Architecture Guide

Continuous Integration, Security Scanning, AI Code Review, Multi-Architecture Builds, and Production Deployment

PROJECT
Django Polls App

DOCUMENT TYPE
Technical Reference

VERSION
1.0

STATUS
Approved

AUTHOR
DevOps Engineering Team

DATE
August 2, 2026

Table of Contents

1 Introduction

This document is the authoritative reference for the **Django Polls App** CI/CD pipeline, implemented with **GitHub Actions**. It explains the architecture, every pipeline stage, security controls, and operational procedures in a format suitable for delivery to clients, managers, and engineering teams.

6	80%	2	3
PIPELINE STAGES	COVERAGE GATE	ARCHITECTURES	SECURITY SCANNERS

What is CI/CD?

CI/CD is an automated software engineering practice that manages the entire release lifecycle. It replaces slow, error-prone manual processes with fast, repeatable automation.

✎

Continuous Integration (CI)

Every code commit is automatically built and verified. Linting, unit tests, and coverage checks run immediately so defects are caught in minutes — not at release time.

⇄

Continuous Delivery (CD)

Code that passes all checks is automatically packaged into a deployable artifact — a signed Docker image — ready for controlled release.

◎

Continuous Deployment

Approved code is released to staging automatically, and to production on version-tag triggers with a manual approval gate.

Why Automation Matters

Benefit	Impact
Velocity	Features reach users in minutes instead of weeks through frequent, automated releases.
Consistency	Every deployment runs identically — no forgotten migrations or missing static assets.

Security	Dependencies and code are scanned automatically on every change; bad code is blocked before it ships.
Feedback	Developers know within minutes whether their change is valid, tested, and secure.

> **i NOTE**
>
> The terms **Continuous Delivery** and **Continuous Deployment** both abbreviate to "CD" but differ in how releases reach production: Delivery requires a manual trigger, while Deployment is fully automatic. This pipeline uses both, per environment.

2 Pipeline Architecture Overview

Trigger Events

The pipeline is executed by GitHub Actions, defined in `.github/workflows/deploy.yml`, and is triggered automatically by:

Event	Trigger	Intended For
Push	Push to `main` or `DevOps`	Validation, security, build, staging deploy
Tag	Semantic version tag (e.g. `v1.2.0`)	Production release
Pull Request	PR targeting `main` / `DevOps`	Validation, security, AI review
Manual	Workflow dispatch from GitHub UI	On-demand pipeline execution

End-to-End Flow

The six stages execute sequentially, with each acting as a quality gate before the next:

1

Validation & Quality Gates

Job: `validate`

- Python linting with **Flake8**
- Django unit & integration tests with coverage
- Minimum **80%** coverage enforced — build aborts below the threshold

2

Compliance & Security Scanning

Job: `security-scans`

- Secret detection with **GitLeaks**
- Dependency vulnerabilities with **Trivy** (fails on HIGH/CRITICAL)
- SAST with **Semgrep** & **CodeQL**, plus SBOM generation

3

AI Code Review

Job: `ai-review` — Pull Requests only

- Analyzes the PR diff with the OpenCode AI engine
- Posts actionable inline recommendations on the PR

4

Multi-Architecture Build & Sign

Job: `build-and-push` — Push events only

- Docker images built for **amd64** and **arm64**
- Pushed to Docker Hub, scanned with Trivy, signed with **Cosign**

5

Staging Deployment

Job: `deploy-staging` — Push to main/DevOps

- Helm upgrade with automatic rollback on failure
- In-cluster smoke tests against `/health/` and `/metrics/`

6

Production Deployment

Job: `deploy-production` — Version tags only

- Manual approval gate via the GitHub production environment
- Controlled Helm release with verification and smoke tests

✓ DECISION LOGIC

Pull Requests execute stages 1–3 (validation, security, AI review). **Pushes** execute stages 1–2 and 4–5 (build and staging). **Version tags** additionally trigger stage 6 (production).

3 Stage 1 — Validation & Quality Gates

This stage enforces code quality before anything else runs. It is the first line of defense and gates all subsequent stages.

What Runs

Check	Command	Failure Behaviour
Linting	`flake8`	**ABORT** on syntax errors / undefined names
Tests	`coverage run manage.py test`	**ABORT** on any failing test
Coverage	`coverage report --fail-under=80`	**ABORT** below 80% coverage

Artifacts Produced

- **HTML coverage report** — uploaded as `coverage-report-html`, retained 14 days.
- **XML coverage report** — uploaded as `coverage-report-xml`, retained 14 days.
- **Job summary** — test status and coverage percentage posted to the GitHub run summary.

> ⚠ **COVERAGE GATE**
>
> The 80% threshold is a hard fail. If a change lowers coverage below 80%, the pipeline terminates immediately and no image is built or deployed.

CHAPTER SUMMARY

Stage 1 guarantees that only linted, tested, sufficiently covered code proceeds. Developers receive feedback in minutes through artifacts and the run summary.

4 Stage 2 — Compliance & Security Scanning

An enterprise-grade security posture is enforced with five complementary scanners that run after validation passes.

Scanner	Category	What It Detects	Gate
GitLeaks	Secrets	Accidental commits of API keys, passwords, SSH keys	Hard fail
Trivy (FS)	Dependencies	Known vulnerabilities in `requirements.txt`	Fail on HIGH/CRIT
Semgrep	SAST	SQL injection, hardcoded config, Django anti-patterns	Report
CodeQL	SAST	Deep, cross-function security analysis	Report
SBOM	Compliance	CycloneDX bill-of-materials of all dependencies	Report

Vulnerability Management Workflow

1. Trivy audits the pinned packages in `requirements.txt` against the latest vulnerability database.
2. If any **HIGH** or **CRITICAL** advisory applies, the build fails with `exit-code: 1`.
3. The scan report is uploaded as `trivy-fs-report` (SARIF format) for inspection.
4. The team upgrades the affected package and re-runs the pipeline.

i SARIF REPORTS

All security findings are emitted in SARIF format, GitHub's standard interchange format, enabling tool-agnostic analysis and future integration with code-scanning dashboards.

✓ REAL INCIDENT

In August 2026, Trivy flagged **Django 5.1.3** with five HIGH/CRITICAL advisories (SQL injection and DoS). The dependency was upgraded to **Django 5.1.14**, after which the scan reported **0 vulnerabilities**.

CHAPTER SUMMARY

Five scanners provide defense-in-depth: secrets, dependencies, source code, and compliance. Critical findings block the release; all reports are archived as artifacts for audit.

5 Stage 3 — AI Code Review

The AI review stage brings automated, intelligent code review directly into the pull-request workflow. It executes only on PR events and is powered by the script `.github/scripts/ai_review.py`.

How It Works

1 Extract the Diff

git diff origin/main...HEAD

- Fetches the target branch and computes the exact change set for the PR.

2 Analyze with AI

OpenCode / Antigravity AI API

- Sends the diff to the AI engine with a senior-engineer review prompt.
- Reviews Dockerfiles, GitHub Actions workflows, Kubernetes configs, and Python/Django code.
- Falls back to a mock review if the API is unreachable.

3 Post the Review

GitHub Issues API

- Comments the review directly on the pull request with a branded banner.
- Emphasizes actionable, production-oriented recommendations.

Review Focus Areas

- **Dockerfile** — multi-stage builds, non-root users, layer caching, pinned base images.
- **GitHub Actions** — pinned action versions, least-privilege permissions, runtime efficiency.
- **Kubernetes / Helm** — liveness & readiness probes, resource limits, security context.
- **Python / Django** — anti-patterns, security vulnerabilities, performance issues.

> ⓘ **OPERATIONAL NOTE**
>
> The review is advisory. Findings never block a merge — the AI supplements human review by flagging issues early in the review loop.

CHAPTER SUMMARY

Stage 3 closes the feedback loop on PRs: the AI acts as an automated second reviewer, surfacing best-practice and security improvements before code is merged.

6 Stage 4 — Build, Scan & Sign

This stage packages the application into a production-ready, multi-architecture container image. It runs on push events and version tags — never on pull requests.

Build Pipeline

Step	Tool	Purpose
Architecture Prep	QEMU + Buildx	Emulate and build for `linux/amd64` and `linux/arm64`
Registry Push	Docker Hub	Publish `harshal001/django-polls-app:latest` plus SHA/tag label
Image Scan	Trivy	Scan image layers; fail on HIGH/CRITICAL (`exit-code: 1`)
Image Signing	Cosign	Sign with GitHub OIDC keyless identity for supply-chain trust

✓ SUPPLY-CHAIN SECURITY

Cosign keyless signing binds the image digest to the GitHub workflow identity via OpenID Connect. This guarantees that only trusted, unmodified images can be deployed — protecting production clusters from tampering.

i TAGGING CONVENTION

When a workflow run is triggered by a version tag, the image is tagged with the tag name (e.g. `v1.2.0`). Otherwise it is tagged with the commit SHA, guaranteeing every deployment is traceable to a commit.

CHAPTER SUMMARY

Stage 4 produces signed, multi-architecture images with cached builds for speed, integrity-protected by Cosign and verified by Trivy before any deployment.

7 Stage 5 — Staging Deployment

Deployment to the staging Kubernetes cluster is fully automated on every push to `main` or `DevOps`.

Deployment Procedure

1. Configure cluster access from the `KUBECONFIG` secret.
2. Install Helm and run `helm upgrade --install` with `values-staging.yaml`.
3. Wait for the release to converge (10-minute timeout).
4. **Auto-rollback** if the upgrade fails — `helm rollback` restores the previous revision.
5. Verify rollout with `kubectl rollout status`.
6. Run in-cluster **smoke tests** against `/health/` and `/metrics/`.
7. Notify the team on Slack with the deployed SHA.

> ⚠ **ROLLBACK STRATEGY**
>
> Failures during `helm upgrade` automatically trigger a rollback to the last known-good revision, and the job exits non-zero so the team is alerted immediately.

CHAPTER SUMMARY

Staging provides a production-like validation environment. Automated rollback and smoke tests ensure broken releases never propagate downstream.

8 Stage 6 — Production Deployment

Production releases are deliberately controlled: they trigger only on semantic version tags and require manual approval.

Release Process

1. Tag

Developer pushes a semantic version tag, e.g. `git tag v1.2.0 && git push origin v1.2.0`.

2. Pipeline

Stages 1–4 run against the tagged commit; the image is built, scanned, and signed.

3. Approval

Deployment waits for a human to approve the run on the `production` GitHub environment.

4. Deploy

Helm deploys with `values-production.yaml`, followed by verification and smoke tests.

Safety Controls

- **Manual approval gate** — the `production` environment guard prevents unintended releases.
- **Tag-only trigger** — arbitrary branch pushes can never reach production.
- **Auto-rollback** — failed upgrades restore the previous revision.
- **Post-deploy verification** — health and metrics endpoints are validated in-cluster.

✓ BEST PRACTICE

Use a strict **semantic versioning** policy (`vMAJOR.MINOR.PATCH`) so that tags are both human-readable and machine-validated for ordering and compatibility.

CHAPTER SUMMARY

Production is protected by layering automation with human control — automated tests and security gates, plus a mandatory approval before the final release.

9 Local Security Scanning Guide

Engineers can reproduce the Trivy dependency scan locally before pushing, shortening the feedback loop.

Install Trivy

```
# One-line installer
curl -sSfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh
-s -- -b /usr/local/bin

# Ubuntu / Debian
sudo apt-get install trivy
```

Run the Scan

```
cd Django-polls-app
trivy fs . --scanners vuln --severity HIGH,CRITICAL
```

Reading the Results

Output	Meaning	Action
0 vulnerabilities	No HIGH/CRITICAL issues	Ready to push
HIGH: n, CRITICAL: n	Advisories found	Upgrade the flagged package, then re-scan

i FIRST RUN

The first scan downloads the vulnerability database (~100 MB). Subsequent scans are fast and incremental.

✓ FASTER FEEDBACK

Run the scan locally on every dependency change. Catching an advisory before the pull request saves a full CI cycle.

10 Operational Procedures & Troubleshooting

Monitoring Reports

- Open the **Actions** tab in GitHub and select a run.
- Download artifacts: coverage reports, Trivy SARIF reports, SBOM.
- Review the **Job Summary** for coverage and test outcomes.

Common Failures and Remedies

Symptom	Root Cause	Remedy
Coverage gate failed	Untested code paths	Inspect HTML report; add tests in `polls/tests.py`
GitLeaks detected secrets	Credential committed	Purge history with `git filter-repo`; store secrets in GitHub Secrets
Trivy scan failed	Outdated dependency	Check `trivy-fs-report`; upgrade the package in `requirements.txt`
Deployment failed	Helm upgrade error	Rollback is automatic; inspect logs and redeploy

11 Conclusion & Key Takeaways

The Django Polls App CI/CD pipeline is a complete, enterprise-grade software delivery system. It combines rigorous quality gates, defense-in-depth security scanning, AI-assisted code review, and tamper-proof container builds with controlled, auditable deployments to staging and production.

5	100%	2	v1.0
SECURITY SCANNERS	AUTOMATED STAGES	DEPLOY ENVIRONMENTS	DOCUMENT VERSION

KEY TAKEAWAYS

- CI catches defects in minutes through linting, testing, and an 80% coverage gate.
- Security is enforced at every layer: secrets, dependencies, source, and image signing.
- AI review accelerates pull-request feedback without blocking merges.
- Staging deploys automatically; production deploys only on approved version tags.
- Every artifact — reports, SBOM, signatures — is retained for audit and traceability.

12 Glossary

Term	Definition
CI	Continuous Integration — automated building and testing of every commit.
CD	Continuous Delivery / Deployment — automated packaging and release of software.
Coverage	The percentage of code executed by automated tests.
SAST	Static Application Security Testing — analysis of source code for vulnerabilities.
SBOM	Software Bill of Materials — inventory of all components in a software product.
SARIF	Static Analysis Results Interchange Format — standard format for scan results.
Multi-Arch	Building container images for multiple CPU architectures (amd64, arm64).
Cosign	A tool for signing and verifying container images, part of the Sigstore project.
Helm	A package manager for Kubernetes that deploys applications via "charts".
OIDC	OpenID Connect — the identity layer used for keyless Cosign signing.
Smoke Test	A quick functional check that a service responds correctly after deployment.